

Proyecto final - Inventario TI y diagnóstico de servicios

Ejercicio práctico integrado

Objetivos	Desarrollar una aplicación Python sencilla, modular y mantenible que lea datos de inventario, valide la información, clasifique equipos y genere informes de soporte.
Material necesario	Terminal, Python 3.13, entorno virtual del curso, Visual Studio Code con extensiones Python y Jupyter, ficheros <code>datos/equipos.csv</code> y <code>datos/politica_servicios.json</code> .

1. Planteamiento del proyecto

Una empresa de servicios TI mantiene un inventario básico de equipos y servicios instalados. El departamento de soporte necesita un pequeño programa en Python que lea esos datos, detecte situaciones operativas relevantes y genere un informe final sin depender de paquetes externos.

El proyecto debe estar pensado como una herramienta interna de diagnóstico. No es una aplicación web ni un sistema completo de monitorización. El objetivo es integrar los conceptos trabajados durante el curso en un ejercicio final realista y asequible.

2. Datos de partida

El alumno recibirá dos ficheros de datos dentro del directorio `datos`:

- `equipos.csv`: inventario de equipos, servicios principales, puertos y métricas básicas.
- `politica_servicios.json`: puertos esperados y umbrales de CPU, memoria y disco.

El CSV contiene menos de 20 registros. Cada fila representa un equipo inventariado por soporte.

```
id,nombre,tipo,sistema,ip,servicio,puerto,estado,cpu,memoria,disco,entorno,responsable
1,srv-web-01,servidor,Linux,10.10.1.10,nginx,443,OK,42,61,55,prod,Equipo Web
```

3. Requisitos funcionales

1. Crear una estructura de proyecto ordenada, con un script principal y un paquete propio para separar responsabilidades.
2. Leer `equipos.csv` usando `csv.DictReader` y convertir los campos numéricos al tipo adecuado.
3. Leer `politica_servicios.json` usando el módulo `json`.

4. **Representar cada equipo mediante una clase Equipo** con atributos, métodos y una representación textual clara.
5. **Validar que la IP tenga formato básico correcto**, que el puerto esté entre 1 y 65535 y que el estado pertenezca a OK, AVISO o ERROR.
6. **Normalizar cadenas de texto**: espacios, mayúsculas/minúsculas y nombres de servicio.
7. **Clasificar cada equipo** como NORMAL, REVISAR o CRITICO según estado y umbrales de CPU, memoria y disco.
8. **Generar un resumen por entorno y por estado** usando diccionarios, listas, conjuntos o tuplas.
9. **Generar un informe de texto** en `salida/informe.txt` con el resumen general y el detalle de equipos críticos o a revisar.
10. **Exportar un fichero** `salida/resumen.json` con los totales calculados.
11. **Registrar la ejecución** en `logs/proyecto.log` usando `logging`.
12. **Gestionar errores previsibles**: ficheros inexistentes, datos incompletos, conversiones inválidas y JSON mal formado.

4. Requisitos técnicos

- No usar paquetes externos (`pandas`, `requests` etc...). El proyecto debe resolverse con Python y su biblioteca estándar.
- Usar `pathlib.Path` para construir rutas.
- Usar **funciones pequeñas** con nombres claros y docstrings breves.
- **Separar cálculo y presentación**: las funciones que calculan no deben depender de `print()`.
- **Evitar variables globales modificables** salvo constantes de configuración.
- **Proteger el punto de entrada** de los módulos con `if __name__ == "__main__":`.
- **Incluir comentarios útiles**, sin repetir literalmente lo que ya dice cada instrucción.
- **Mantener compatibilidad con ejecución** desde terminal y revisión desde notebook.

5. Estructura mínima esperada

```
Proyecto_Final_Python_Inventario_TI/
├── datos/
│   ├── equipos.csv
│   └── politica_servicios.json
├── inventario_ti/
│   ├── __init__.py
│   ├── modelos.py
│   ├── validaciones.py
│   ├── io_datos.py
│   ├── analisis.py
│   └── reportes.py
├── logs/
├── salida/
├── main.py
└── README.md
```

6. Tareas del alumno

1. **Preparar el entorno virtual** del proyecto y comprobar la versión de Python.
2. **Revisar los ficheros de datos** y entender qué representa cada columna.
3. **Crear el paquete** `inventario_ti` y distribuir el código en módulos.
4. **Implementar la clase Equipo** y sus métodos de ayuda.
5. **Implementar funciones de lectura** y validación de datos.
6. **Implementar funciones** de análisis, clasificación y resumen.
7. **Implementar funciones** de generación de informes.
8. **Ejecutar** `main.py` desde la terminal y revisar los ficheros generados.

7. Resultado esperado

Al finalizar, el proyecto debe permitir ejecutar el diagnóstico con un comando similar al siguiente:

```
python main.py --datos datos/equipos.csv --politica datos/politica_servicios.json
```

La ejecución debe crear los directorios `salida` y `logs` **si no existen, generar** `informe.txt`, `resumen.json` **y registrar los eventos principales en** `proyecto.log`.

El informe final debe mostrar como mínimo: número total de equipos, entornos detectados, servicios únicos, totales por estado, totales por entorno y una lista de equipos clasificados como REVISAR o CRITICO.

8. Criterios de revisión

- El programa se ejecuta sin errores con los datos entregados.
- La estructura del proyecto es clara y los módulos tienen responsabilidades diferenciadas.
- El código usa funciones, clase, colecciones, gestión de errores, entrada/salida, módulos propios y biblioteca estándar.
- El informe generado es legible y útil para un técnico de soporte.
- El notebook permite seguir el razonamiento paso a paso sin bloquearse con `input()`.

9. Ampliaciones opcionales

- Añadir un argumento `--solo-criticos` para mostrar únicamente equipos críticos.
- Exportar también un CSV con los equipos clasificados.
- Crear una pequeña base SQLite con el inventario procesado.
- Añadir pruebas manuales con datos incorrectos para comprobar la gestión de errores.